A PROJECT REPORT ON

PREDICTIVE MAINTENANCE OF INDUSTRIAL HYDRAULIC SYSTEMS USING FEATURE ENGINEERING

Submitted in partial fulfilment of the requirements
for the Capstone Project in Feature Engineering

Submitted by

Sahil Khan Pathan	24AD1199
Vedant Mali	24AD1145
Sejal Zende	24AD1158

Under the guidance of

Prof. _____

Department of Artificial Intelligence and Data Science Engineering

DY Patil University

Navi Mumbai, Maharashtra, India

Academic Year 2026–2027

CERTIFICATE

This is to certify that the project report entitled **"Predictive Maintenance of Industrial Hydraulic Systems Using Feature Engineering"** is a bonafide record of work carried out by

Sahil Khan Pathan	(24AD1199)
Vedant Mali	(24AD1145)
Sejal Zende	(24AD1158)

in partial fulfilment of the requirements of the Capstone Project in Feature Engineering, Department of Artificial Intelligence and Data Science Engineering, DY Patil University, Navi Mumbai, during the academic year 2026–2027. The work presented has not been submitted elsewhere for the award of any degree or diploma.

_____

Project Guide
Prof. _____

Date: _____ Place: Navi Mumbai

DECLARATION

We hereby declare that the project report entitled **"Predictive Maintenance of Industrial Hydraulic Systems Using Feature Engineering"** is our own work, carried out under the guidance of our project guide. The dataset used is the publicly available UCI *Condition Monitoring of Hydraulic Systems* dataset, and all external sources have been acknowledged in the references. Every numerical result in this report was produced by executing the project code on that dataset; no value has been copied from the literature.

Sahil Khan Pathan (24AD1199) _____

Vedant Mali (24AD1145) _____

Sejal Zende (24AD1158) _____

Date: _____ Place: Navi Mumbai

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide for continuous guidance, constructive criticism and encouragement throughout this project. We are thankful to the Head of the Department of Artificial Intelligence and Data Science Engineering and to all faculty members of DY Patil University, Navi Mumbai, for providing the academic environment and resources that made this work possible.

We also thank the Centre for Mechatronics and Automation Technology (ZeMA gGmbH), Saarbrücken, and the UCI Machine Learning Repository for making the hydraulic condition-monitoring dataset publicly available, and the developers of the open-source Python ecosystem—NumPy, pandas, SciPy, scikit-learn, XGBoost, Matplotlib and Streamlit—on which this project is built.

Finally, we thank our families and friends for their constant support.

Sahil Khan Pathan
Vedant Mali
Sejal Zende

ABSTRACT

Hydraulic power units drive presses, injection-moulding machines and mobile equipment, and their unplanned failure halts whole production lines. Predictive maintenance aims to detect component degradation from routinely measured signals before failure occurs. This project builds a condition monitoring system for an industrial hydraulic test rig using the UCI *Condition Monitoring of Hydraulic Systems* dataset: 2,205 operating cycles of 60 seconds, each recorded by 17 sensors at 100 Hz, 10 Hz and 1 Hz, with four labelled component conditions—cooler, valve, internal pump leakage and hydraulic accumulator.

One cycle contains 43,680 raw values, far more than the number of cycles, and the channels are not time-aligned. The core of the project is therefore **feature engineering**: each cycle is converted into 626 physically interpretable features organised in five blocks—statistical, time-domain, rolling-window, frequency-domain and hydraulic domain-specific features such as pressure differentials, hydraulic power and electro-hydraulic efficiency. Unsupervised cleaning reduces them to 553, and a three-stage selection chain (correlation filter, mutual information and tree importance) reduces them to 40 per model. All learned steps are placed inside a single scikit-learn pipeline to prevent data leakage.

Three representations—a 17-feature baseline of per-sensor means, the full engineered set and the selected set—are compared across four classifiers. Engineered features raise the cross-validated macro-F1 of logistic regression on the valve from 0.672 to 0.991 and on the accumulator from 0.642 to 0.977, and the best models reach a held-out macro-F1 of 1.000 for the cooler, valve and pump and 0.988 for the accumulator. A temporally blocked cross-validation gives more conservative estimates of 0.995, 0.984, 0.968 and 0.916. The trained pipelines are deployed in an interactive Streamlit dashboard that predicts component health for any cycle and explains each prediction.

Keywords: predictive maintenance, feature engineering, condition monitoring, hydraulic systems, feature selection, machine learning.

Contents

Certificate ii
Declaration iii
Acknowledgement iv
Abstract v

1 Introduction **1**
1.1 Background . . . 1
1.2 Problem Statement . . . 1
1.3 Why Feature Engineering Is the Focus . . . 1
1.4 Objectives . . . 1
1.5 Scope and Limitations . . . 2
1.6 Organisation of the Report . . . 2
1.7 Literature Review . . . 2

2 Dataset Description **4**
2.1 Source and Structure . . . 4
2.2 Target Variables . . . 4

3 Feature Engineering Methodology **7**
3.1 Unit of Analysis and Design Principles . . . 7
3.2 Block A: Statistical Features (251) . . . 7
3.3 Block B: Time-Domain Features (162) . . . 7
3.4 Block C: Rolling-Window Features (54) . . . 8
3.5 Block D: Frequency-Domain Features (63) . . . 8
3.6 Block E: Hydraulic Domain Features (96) . . . 8
3.7 Feature Cleaning . . . 9
3.8 Feature Selection Chain . . . 9
3.9 Leakage-Safe Pipeline . . . 10

4 Experiments and Results **11**
4.1 Three Feature Experiments . . . 11
4.2 Classifiers . . . 11
4.3 Evaluation Protocol . . . 11
4.4 Effect of Feature Engineering . . . 11
4.5 Which Feature Blocks Were Selected . . . 12
4.6 Which Features Matter and Why . . . 13
4.7 Held-Out Test Performance and Honest Evaluation . . . 14
4.8 Error Analysis . . . 15

5 Dashboard Implementation **16**

6 Conclusion and Future Scope **18**
6.1 Conclusion . . . 18
6.2 Future Scope . . . 18

References **19**

List of Figures

2.1 One 60-second operating cycle as seen by PS1, PS3, EPS1, FS1, TS1 and CE . . . 5
2.2 Dashboard: dataset summary and sensor table . . . 6

3.1 Dashboard: feature engineering funnel from raw samples to model inputs . . . 9
3.2 Dashboard: feature blocks, their size and what each captures . . . 10
3.3 Feature count at each selection stage for the valve model . . . 10

4.1 Cross-validated macro-F1 of each classifier on the baseline (Base), engineered (Eng.) and selected (Sel.) representations 12
4.2 Dashboard: the logistic-regression gain from engineered features for each component 12
4.3 Selected features per engineering block for the valve model (left) and selection rate relative to block size (right) 13
4.4 Dashboard: selected model, feature set and number of features reaching each classifier 13
4.5 Top ten features of the valve model by random-forest importance 14
4.6 Dashboard: realistic (blocked) versus optimistic (random) performance 15
4.7 Blocked CV, random CV and test macro-F1 of each final model (y-axis starts at 0.88) 15
4.8 Test-set confusion matrix of the accumulator model 15

5.1 Dashboard: overview tab with system check in the sidebar 16
5.2 Dashboard: component status cards and comparison with the recorded condition . 17
5.3 Dashboard: explainability panel for the cooler model 17

List of Tables

2.1 Sensors, sampling rates and samples per cycle 4
2.2 Target variables and class distribution 5

3.1 Hydraulic domain features 9

4.1 Best cross-validated macro-F1 per representation 12
4.2 Origin of the 40 selected features by engineering block 13
4.3 Final models: test set and temporally blocked cross-validation 14

CHAPTER 1

INTRODUCTION

1.1 Background

Hydraulic systems convert electrical power into high-force linear and rotary motion through a pump, valves, an accumulator and a cooling circuit. Their components wear gradually: coolers lose efficiency as they foul, valves develop switching lag, pumps begin to leak internally and accumulators lose their gas pre-charge. Traditional maintenance is either *reactive* (repair after failure) or *preventive* (service at fixed intervals), and both are costly—the first through unplanned downtime, the second through replacing healthy parts. **Predictive maintenance** instead estimates component health from sensor data while the machine runs, so that service is performed only when degradation is actually detected.

1.2 Problem Statement

Given the raw sensor recordings of one 60-second operating cycle of a hydraulic test rig, predict the condition level of four components—cooler, valve, pump and accumulator—and present the result in a form that an engineer can check and act on. Each target is a multiclass classification problem with three or four documented condition levels.

1.3 Why Feature Engineering Is the Focus

A single cycle contains 43,680 raw values, but the dataset has only 2,205 cycles. Using raw samples as features puts any model in an extreme "more columns than rows" regime. The sensors also run at three sampling rates, so column k of a pressure signal and column k of a temperature signal do not even refer to the same instant. Most importantly, faults are physical: a leaking pump changes the relationship between motor power and delivered flow, and a degraded cooler changes temperature differences. These relationships exist *between* channels, and no per-sample representation exposes them. Carefully engineered features solve all three problems at once.

1.4 Objectives

1. To study the hydraulic rig dataset and its sensors, sampling rates and condition labels.
2. To design a feature engineering pipeline that converts each raw cycle into a compact, physically interpretable feature vector.
3. To reduce the feature set systematically through cleaning and leakage-safe feature

selection.

4. To measure the contribution of feature engineering by comparing a baseline, the full engineered set and the selected set across four classifiers.
5. To evaluate the final models honestly on a held-out test set and with a temporally blocked cross-validation.
6. To deploy the models in an interactive dashboard with explainability.

1.5 Scope and Limitations

The project classifies the *current* condition of each component. The dataset contains condition levels but no failure timestamps, so Remaining Useful Life (RUL) cannot be estimated and is not claimed. All data come from a single laboratory rig with deliberately set fault levels.

1.6 Organisation of the Report

Section 1.7 reviews related work. Chapter 2 describes the dataset. Chapter 3, the core of the report, presents the feature engineering and feature selection methodology. Chapter 4 describes the experiments and results, Chapter 5 the dashboard, and Chapter 6 concludes.

1.7 Literature Review

Hydraulic condition monitoring. Helwig, Pignanelli and Schütze [1] introduced the test rig used in this project and classified component conditions with linear discriminant analysis on statistical and spectral features extracted from each cycle. They showed that compact cycle statistics carry most of the discriminative information and reported lower classification rates for the accumulator than for other components—an observation this project also confirms.

Machine learning for predictive maintenance. Carvalho *et al.* [3] systematically reviewed machine learning in predictive maintenance and found random forests, neural networks, support vector machines and k-means to be the most frequently used methods. Tree ensembles such as random forest [7] and gradient boosting (XGBoost) [8] are strong on tabular data because they learn feature interactions automatically.

Time-series feature extraction. Libraries such as *tsfresh* [4] extract hundreds of generic statistics from time series and filter them with hypothesis tests. This project follows the same idea—many candidate features, then systematic filtering—but adds domain features derived from hydraulic physics that generic libraries cannot produce.

Feature selection. Guyon and Elisseeff [5] distinguish filter, wrapper and embedded methods. The selection chain in this project combines two filters (correlation and mutual

information [6]) with an embedded method (tree importance).

Evaluation pitfalls. Kaufman *et al.* [10] describe how leakage of information from evaluation data inflates results, and Roberts *et al.* [11] recommend blocked cross-validation for temporally dependent data. Both ideas shape the evaluation design used here.

CHAPTER 2

DATASET DESCRIPTION

2.1 Source and Structure

The dataset was recorded by ZeMA gGmbH, Saarbrücken, and is published in the UCI Machine Learning Repository [2]. The rig consists of a primary working circuit and a secondary cooling-filtration circuit connected through the oil tank. It repeats a constant 60-second load cycle while the condition of four components is varied. Each sensor is stored as one text file in which every row is one cycle. There are **2,205 cycles** and no missing values.

Table 2.1: Sensors, sampling rates and samples per cycle

Sensor(s)	Physical quantity	Unit	Rate [Hz]	Samples/cycle
PS1–PS6	Pressure	bar	100	6000
EPS1	Motor power	W	100	6000
FS1, FS2	Volume flow	l/min	10	600
TS1–TS4	Temperature	°C	1	60
VS1	Vibration	mm/s	1	60
CE	Cooling efficiency (virtual)	%	1	60
CP	Cooling power (virtual)	kW	1	60
SE	Efficiency factor (virtual)	%	1	60
Total raw values per cycle				**43,680**

Figure 2.1 shows one cycle as recorded by six channels. The load programme produces step changes in pressure, motor power and flow roughly every 10 seconds, while temperatures and cooling efficiency drift slowly. The very different shapes and time scales of these signals are the reason each channel needs its own set of descriptors.

2.2 Target Variables

Each cycle carries four condition labels (Table 2.2). Three targets are imbalanced, so macro-F1, which weights every class equally, is used as the main metric. A separate *stable flag* marks 756 cycles in which static conditions may not have been reached; it describes the rig rather than a component and is kept as metadata.

The dashboard's Dataset tab presents the same information interactively (Fig. 2.2), including a class-distribution chart for each component coloured by documented severity.

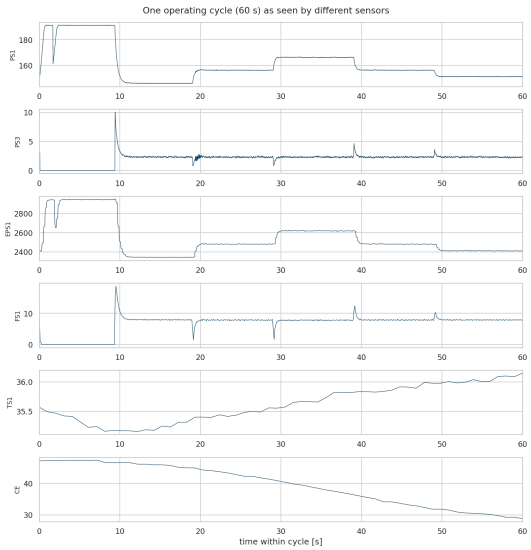


Figure 2.1: One 60-second operating cycle as seen by PS1, PS3, EPS1, FS1, TS1 and CE

Table 2.2: Target variables and class distribution

Component	Condition levels (cycle count)	Classes	Imbalance
Cooler [%]	3 close to total failure (732), 20 reduced efficiency (732), 100 full efficiency (741)	3	1.01
Valve [%]	73 close to total failure (360), 80 severe lag (360), 90 small lag (360), 100 optimal (1125)	4	3.13
Pump leakage	0 none (1221), 1 weak (492), 2 severe (492)	3	2.48
Accumulator [bar]	90 close to total failure (808), 100 severely reduced (399), 115 slightly reduced (399), 130 optimal (599)	4	2.03

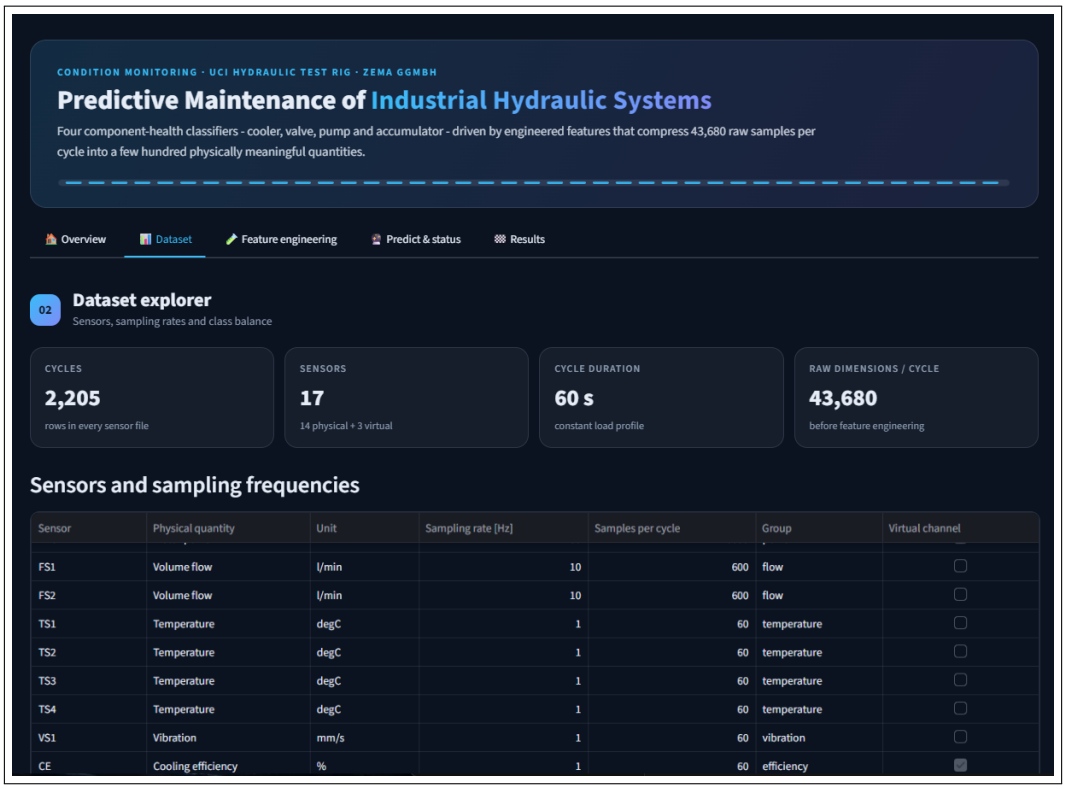


Figure 2.2: Dashboard: dataset summary and sensor table

CHAPTER 3

FEATURE ENGINEERING METHODOLOGY

3.1 Unit of Analysis and Design Principles

One row of the engineered dataset is one complete 60-second cycle. Every feature is computed from that cycle alone, so the transformation can be applied identically to a new cycle at prediction time. Four principles guided the design:

- **Respect sampling rates.** Window lengths and frequency features are defined in physical units (seconds, hertz), not in sample counts.
- **Describe shape, not samples.** Statistics summarise how a signal behaves over the cycle rather than its value at a particular sample.
- **Encode physics.** Cross-channel features express hydraulic relationships that a fault changes.
- **Stay interpretable.** Every feature has a name an engineer can read, such as `dp_PS1_PS2_mean` or `efficiency_PS1_FS1`.

3.2 Block A: Statistical Features (251)

For each sensor signal $x_1, \ldots, x_n$ within one cycle, 15 descriptive statistics are computed: mean, median, standard deviation, variance, minimum, maximum, range, root mean square, mean absolute value, skewness, kurtosis, first and third quartiles, interquartile range and the coefficient of variation. The coefficient of variation is skipped for the four temperature channels, because a Celsius reading has an arbitrary zero and its CV has no physical meaning ($17 \times 15 - 4 = 251$). For example,

$$\mathrm{RMS} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2}, \qquad \mathrm{CV} = \frac{\sigma}{|\mu|}. \tag{3.1}$$

Degradation often changes the *distribution* of a signal before its average. A valve with switching lag spends longer in transition, which widens the pressure distribution even when mean pressure is unchanged. Skewness and kurtosis capture this asymmetry and peakedness.

3.3 Block B: Time-Domain Features (162)

These describe how a signal evolves during the cycle: peak-to-peak value, peak amplitude, signal energy $\frac{1}{n}\sum x_i^2$, statistics of the first difference $\Delta x_i = x_{i+1} - x_i$ (mean, standard devia-

tion, mean and maximum absolute change), the linear slope over the cycle and the number of zero crossings of the mean-removed signal. For channels sampled at 10 Hz or faster, the number of peaks is also counted, where a peak is a local maximum above $\mu + \sigma$ so that ripple on a flat plateau is not counted. The first-difference statistics are sensitive to how sharply a signal switches, which makes them useful for valve lag.

3.4 Block C: Rolling-Window Features (54)

A moving window captures local behaviour that whole-cycle statistics average out. The window is fixed at **one second** and converted into samples from each sensor's own rate:

$$w = \max\big(3,\ \mathrm{round}(1\,\mathrm{s} \times f_s)\big), \tag{3.2}$$

giving 100 samples for 100 Hz channels and 10 for 10 Hz channels. Within the window, the rolling mean and rolling standard deviation are computed, and their spread, range, minimum, maximum, mean and maximum over the cycle become features (six per sensor). The block is applied only to the nine channels sampled at 10 Hz or faster; a 1 Hz channel has only 60 points, too few for a meaningful window.

3.5 Block D: Frequency-Domain Features (63)

For the same nine fast channels, the DC component is removed and a Fast Fourier Transform is computed. Seven compact features are kept: dominant frequency and its magnitude, log spectral energy, spectral centroid and the relative energy in three frequency bands. The band edges are **logarithmically spaced** between the fundamental ($1/60$ Hz) and the Nyquist limit. With linear bands over 0–50 Hz, almost all energy would fall in the first band because the load programme changes on a scale of tens of seconds, and the other bands would be constant. Log spacing gives resolution where the hydraulic dynamics actually live.

3.6 Block E: Hydraulic Domain Features (96)

This block encodes relationships between channels that are directly linked to component faults (Table 3.1).

For instance, a leaking pump draws similar electrical power but delivers less flow, so the specific power and efficiency features shift even when each raw signal looks normal on its own.

Table 3.1: Hydraulic domain features

Feature group	Definition	Fault it targets
Pressure differentials and ratios	$\bar{p}_a - \bar{p}_b$ and $\bar{p}_a/\bar{p}_b$ for all 15 pairs of PS1–PS6; also differences of their standard deviations	Valve lag, leakage between circuit points
Flow relationships	$\overline{FS1} - \overline{FS2}$, ratio, relative imbalance, relative variability	Pump leakage
Specific power	$\overline{EPS1}/\overline{FS}$ (electrical power per unit flow)	Pump leakage
Hydraulic power	$P_h[\mathrm{kW}] = \bar{p}[\mathrm{bar}] \cdot \bar{Q}[\mathrm{l/min}]/600$	Pump, valve
Electro-hydraulic efficiency	$\eta = 1000\,P_h/\overline{EPS1}$	Pump wear
Hydraulic resistance	$\bar{p}/\overline{FS1}$	Valve, accumulator
Temperature relations	pairwise ΔT, spread, mean, maximum rise	Cooler
Cooling conductance	$\overline{CP}/\lvert\Delta T\rvert$, $\overline{CP}/\overline{CE}$, cooling share of power	Cooler
Vibration relations	vibration per unit power, relative vibration variability	Pump, mechanical wear

3.7 Feature Cleaning

The five blocks produce **626 features**, a 70-fold compression of the 43,680 raw values. Before any learning, three label-free checks remove uninformative columns: 4 *constant* features (for example `PS2_min`), 4 *near-zero-variance* features (for example `PS5_slope`) and 65 *exact duplicates* (for example `PS1_peak_to_peak`, identical to `PS1_range`). This leaves **553 features**. Because these checks never look at the labels, running them on all cycles cannot leak target information.

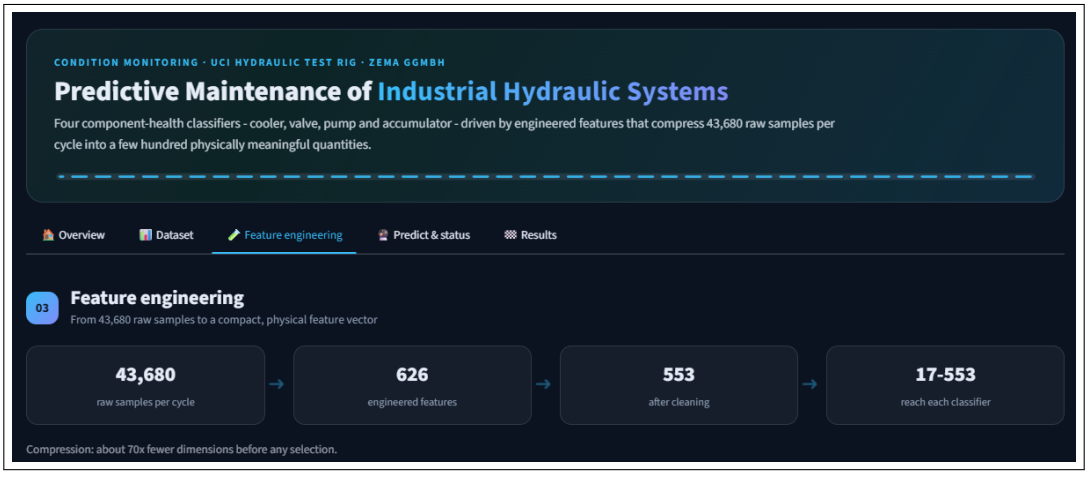


Figure 3.1: Dashboard: feature engineering funnel from raw samples to model inputs

3.8 Feature Selection Chain

Not every one of the 553 features is useful for every component, and redundant features slow down and confuse some models. The *selected* representation applies three stages in

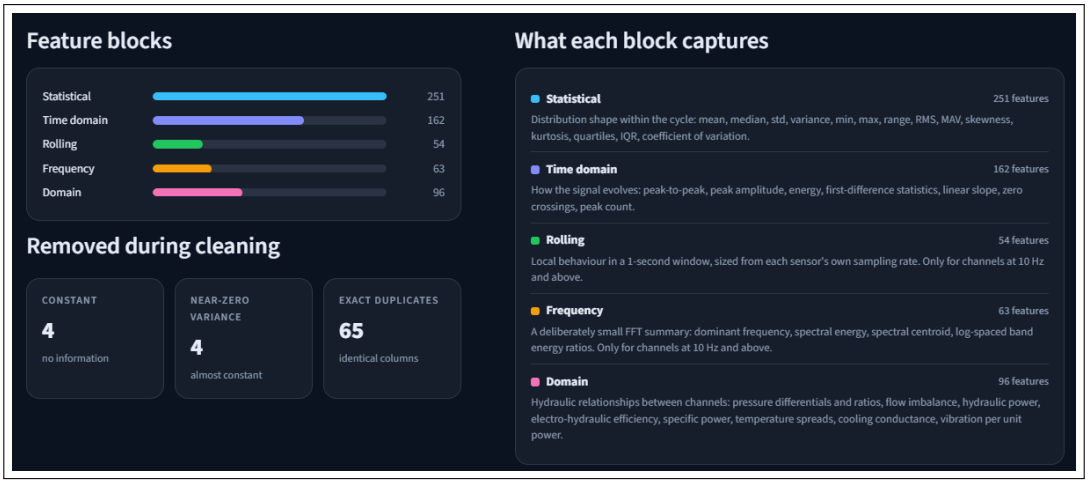


Figure 3.2: Dashboard: feature blocks, their size and what each captures

sequence:

1. **Correlation filter** ($|r| > 0.95$): of any pair of nearly identical features, the later one is dropped. This removes redundancy and reduces the valve set from 553 to 238.
2. **Mutual information**: the top 50 % of features ranked by $I(X_j;Y)$, a model-free measure of non-linear dependence between a feature and the target (238 $\rightarrow$ 119).
3. **Tree importance**: a 200-tree random forest ranks the survivors by impurity decrease, and the top 40 are kept (119 $\rightarrow$ 40). Unlike the univariate filter, this judges each feature in the context of the others.

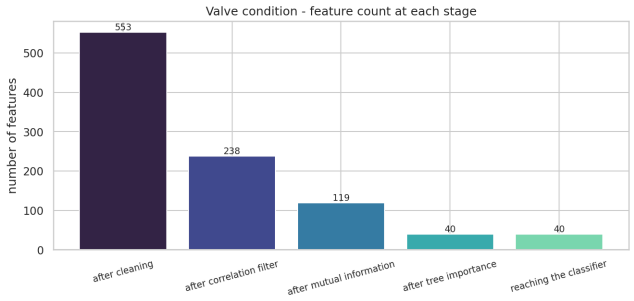


Figure 3.3: Feature count at each selection stage for the valve model

3.9 Leakage-Safe Pipeline

Selection is a *learned* step: it uses the labels. If it were fitted once on all data, the validation folds would have influenced which features were chosen, and scores would be optimistic. To prevent this, imputation, scaling, the three selection stages and the classifier are all stages of one scikit-learn `Pipeline` [9]. When the pipeline is fitted on a training fold, every stage is re-estimated from that fold alone.

CHAPTER 4

EXPERIMENTS AND RESULTS

4.1 Three Feature Experiments

To measure what feature engineering contributes, three representations are compared for every component:

- **Baseline**: the per-sensor cycle mean only (17 features), the simplest summary a practitioner might try first;
- **Engineered**: all 553 cleaned engineered features;
- **Selected**: the engineered features passed through the selection chain (40 features).

4.2 Classifiers

Four classifiers with different inductive biases are used, with fixed hyperparameters so that differences in score come from the features rather than from tuning: **logistic regression** (linear reference), **random forest** (300 trees), **gradient boosting** (XGBoost, 300 trees, depth 5, learning rate 0.1) and an **RBF support vector machine** ($C = 10$). For the imbalanced targets, classes are weighted inversely to their frequency. Logistic regression is the most informative model for this study: it cannot build new features internally, so its score directly shows how linearly separable a representation is.

4.3 Evaluation Protocol

For each component the cycles are split into 80 % training (1,764) and 20 % test (441), stratified by class. All 12 combinations of representation and model are scored with stratified 5-fold cross-validation on the training set only. The best configuration by mean macro-F1 is refitted and evaluated exactly once on the test set. Because neighbouring cycles are nearly identical, a **temporally blocked cross-validation** (blocks of 25 consecutive cycles held out together) is also run as a leakage check.

4.4 Effect of Feature Engineering

Table 4.1 shows the best cross-validated macro-F1 for each representation, and Fig. 4.1 shows every model.

The effect is largest where the baseline is weakest. For the **valve**, logistic regression rises from 0.672 to 0.991 and the SVM from 0.529 to 0.989. For the **accumulator**, logistic regression rises from 0.642 to 0.977. These jumps show that the engineered features make

Table 4.1: Best cross-validated macro-F1 per representation

Component	Baseline	Engineered	Selected	Gain (Eng.)	Gain (Sel.)
Cooler	0.9983	0.9977	0.9983	−0.0006	0.0000
Valve	0.9682	0.9925	0.9946	+0.0243	+0.0263
Pump	0.9906	0.9946	0.9954	+0.0039	+0.0048
Accumulator	0.9706	0.9922	0.9911	+0.0216	+0.0206

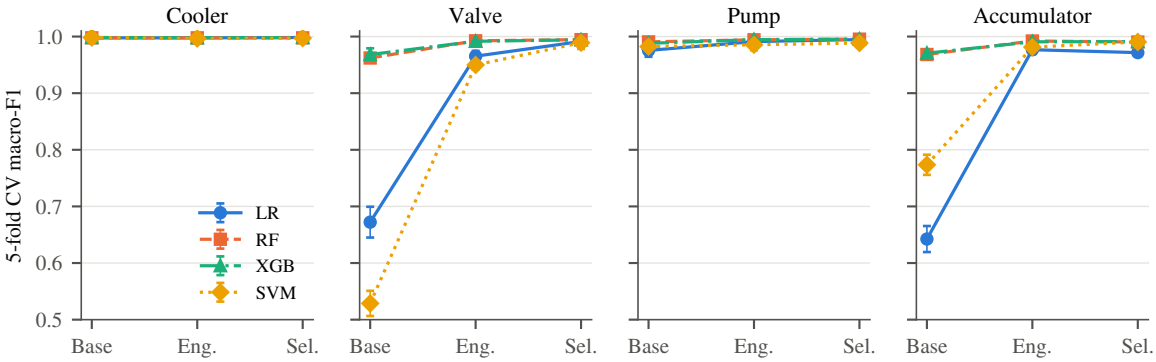


Figure 4.1: Cross-validated macro-F1 of each classifier on the baseline (Base), engineered (Eng.) and selected (Sel.) representations

the classes nearly linearly separable. The **cooler** is already solved by the sensor means (0.998), because the cooling-related virtual channels separate its three levels directly, so engineering adds nothing there. Tree ensembles gain less because they build non-linear combinations internally, but they still improve by 2–3 points on the valve and accumulator.

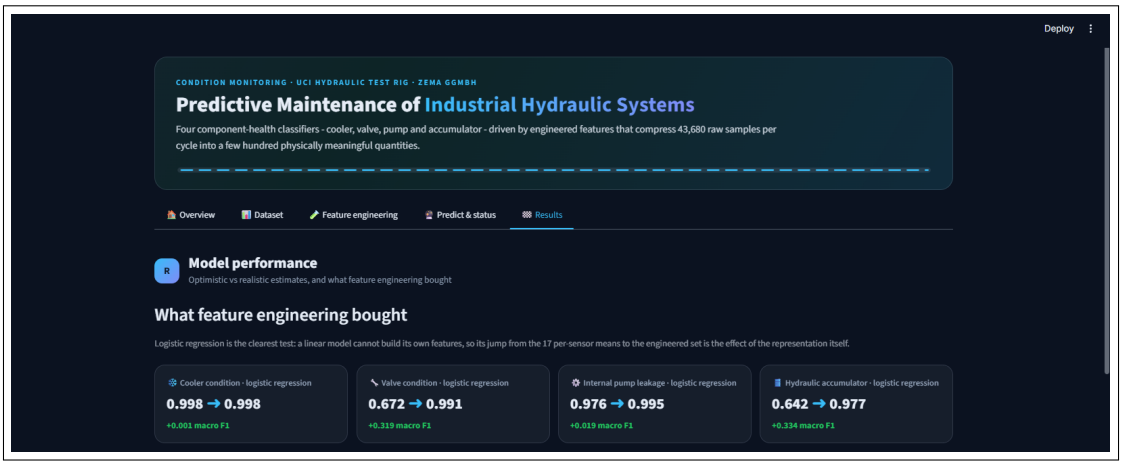


Figure 4.2: Dashboard: the logistic-regression gain from engineered features for each component

4.5 Which Feature Blocks Were Selected

Table 4.2 shows where the 40 selected features of the valve and pump models came from. The valve model draws on every block, led by time-domain features that measure how sharply pressure switches, and frequency features. The pump model relies mostly on statistical features of flow and motor power, plus domain features. Relative to block size,

frequency features had the highest selection rate for the valve (Fig. 4.3).

Table 4.2: Origin of the 40 selected features by engineering block

Model	Statistical	Time-domain	Rolling	Frequency	Domain
Valve	9	11	5	8	7
Pump	20	6	3	2	9

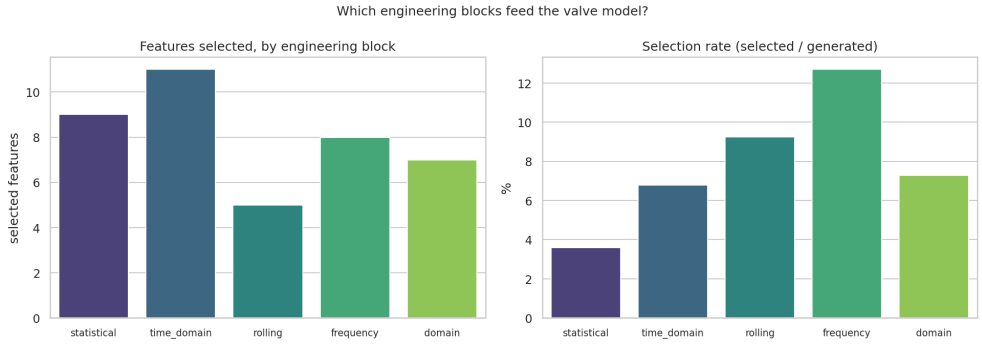


Figure 4.3: Selected features per engineering block for the valve model (left) and selection rate relative to block size (right)

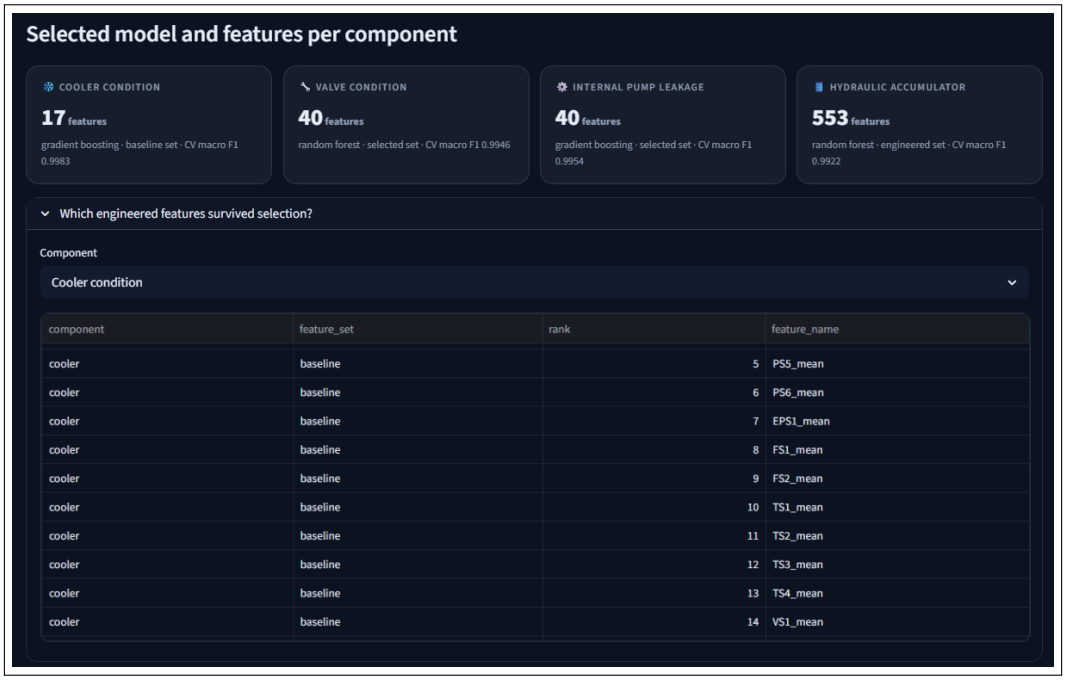


Figure 4.4: Dashboard: selected model, feature set and number of features reaching each classifier

4.6 Which Features Matter and Why

Figure 4.5 shows the ten most important features of the valve model. The top feature, `PS2_cv`, is the coefficient of variation of pressure PS2; switching lag changes how much pressure fluctuates during the cycle. It is followed by a frequency band ratio, first-difference

features of PS2 and the domain feature `dp_PS1_PS2_mean`, the pressure drop between two circuit points. For the pump, permutation importance on the test set is dominated by `FS1_q75` (0.443) and `EPS1_max` (0.093): internal leakage reduces delivered flow for the same motor power. For the cooler, the virtual cooling power CP and cooling efficiency CE dominate. Each of these rankings has a direct physical explanation, which is the main benefit of interpretable engineered features.

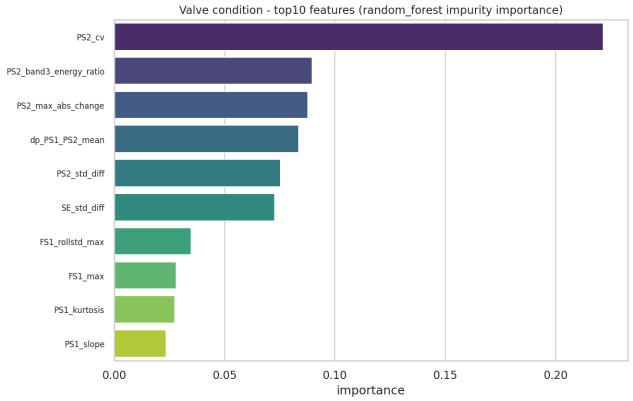


Figure 4.5: Top ten features of the valve model by random-forest importance

4.7 Held-Out Test Performance and Honest Evaluation

Table 4.3 gives the single test-set evaluation of each selected model together with the blocked cross-validation score.

Table 4.3: Final models: test set and temporally blocked cross-validation

Component	Model	Features	Test acc.	Test F1	Blocked CV F1	Gap
Cooler	XGBoost	Baseline (17)	1.000	1.000	0.995	0.003
Valve	Random forest	Selected (40)	1.000	1.000	0.984	0.011
Pump	XGBoost	Selected (40)	1.000	1.000	0.968	0.027
Accumulator	Random forest	Engineered (553)	0.989	0.988	0.916	0.076

Gap = random 5-fold CV macro-F1 minus blocked CV macro-F1.

The test scores are perfect for three components, but the test set was drawn by the same random procedure as the cross-validation folds, so neighbouring, near-identical cycles can appear on both sides. The blocked cross-validation removes this advantage and is the more realistic estimate (Fig. 4.7); the dashboard's Results tab shows the same comparison (Fig. 4.6). The gap is small for the cooler and valve and largest for the accumulator (7.6 points), which is therefore the hardest component.

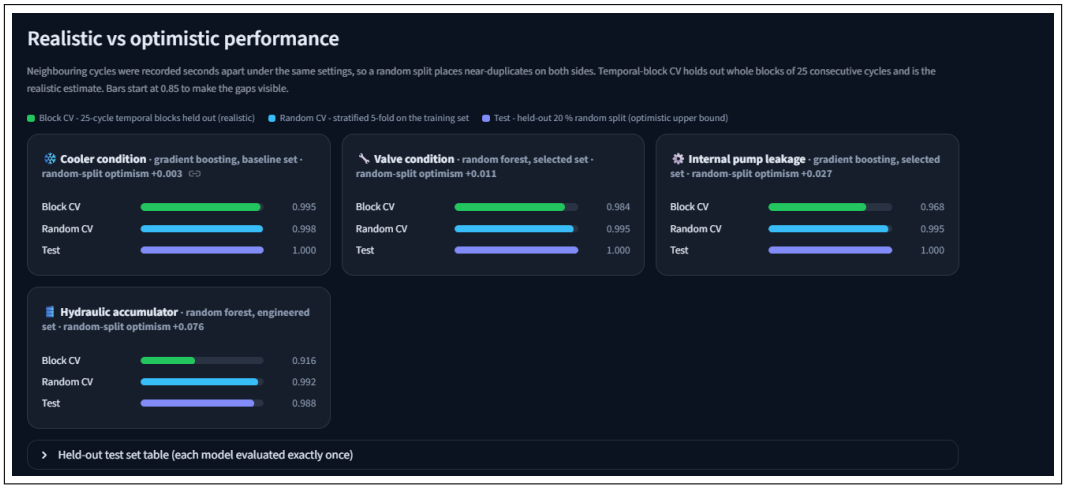


Figure 4.6: Dashboard: realistic (blocked) versus optimistic (random) performance

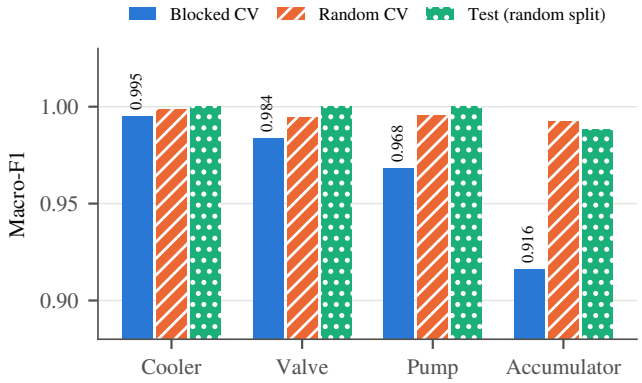


Figure 4.7: Blocked CV, random CV and test macro-F1 of each final model (y-axis starts at 0.88)

4.8 Error Analysis

The only test errors are five accumulator cycles with a true pre-charge of 100 bar predicted as 90 bar (Fig. 4.8). These are the two lowest pre-charge levels, whose pressure transients differ only slightly. The error is conservative from a maintenance point of view: it overstates degradation, so it would trigger an early inspection rather than a missed failure.

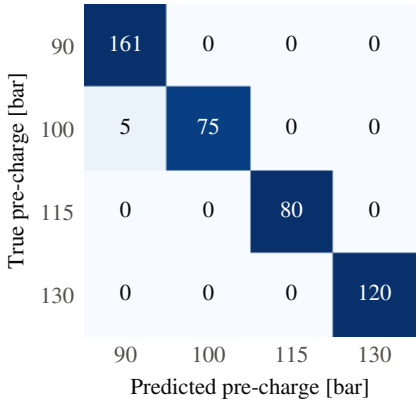


Figure 4.8: Test-set confusion matrix of the accumulator model

CHAPTER 5

DASHBOARD IMPLEMENTATION

The trained pipelines are served by a Streamlit web dashboard. The dashboard never retrains: it loads the saved pipelines, so each new cycle is transformed by exactly the same feature engineering, cleaning and selection used in training. It has five tabs—Overview, Dataset, Feature engineering, Predict & status, and Results—and a sidebar that confirms the dataset and models are available (Fig. 5.1).

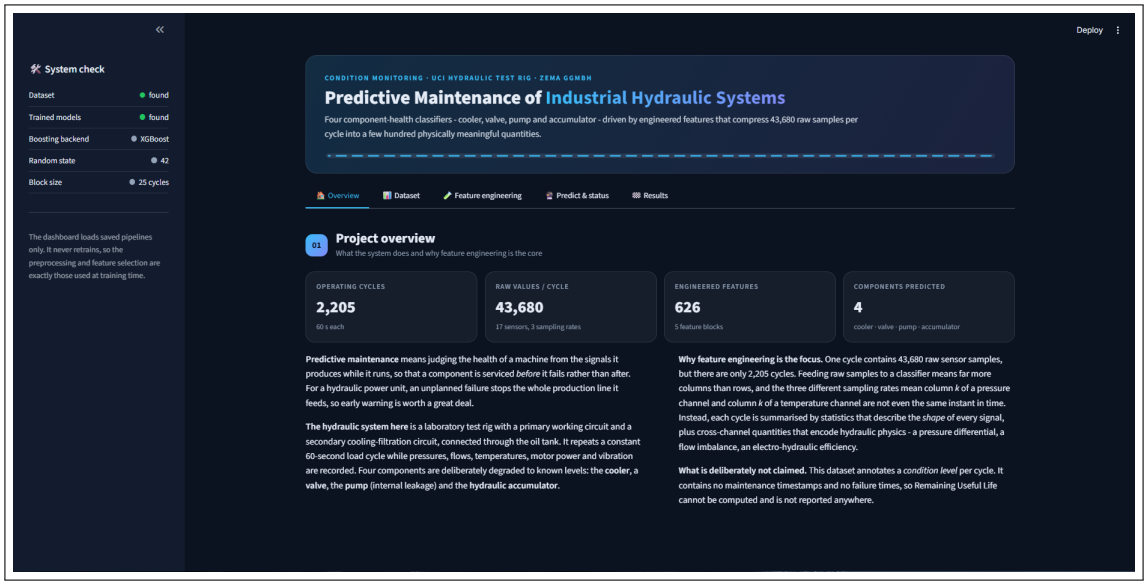


Figure 5.1: Dashboard: overview tab with system check in the sidebar

Prediction. The user selects any cycle, picks a random one or uploads a raw-sensor CSV file. The app extracts the 626 features, scores the four models and shows an overall system status equal to the worst component status. Each component card gives the predicted level, the documented meaning, the severity and the model confidence. For dataset cycles the prediction is compared with the recorded label, and the app states whether the cycle was part of each model's training or held-out test set (Fig. 5.2).

Explainability. For the chosen component, the app lists the features the model relies on most, together with the value each feature took in the current cycle, and plots the raw sensor traces (Fig. 5.3). This lets an engineer check an alarm against the physics before acting on it.

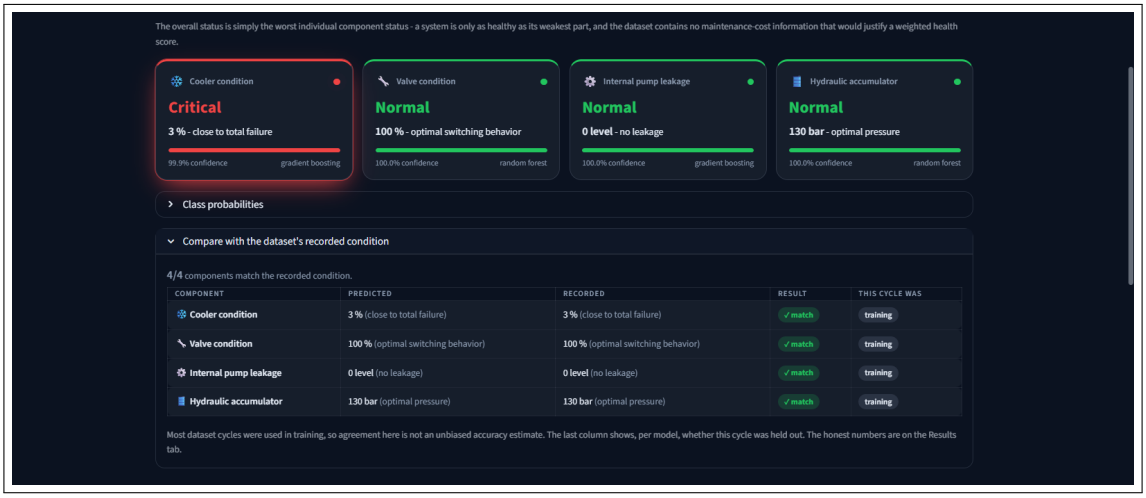


Figure 5.2: Dashboard: component status cards and comparison with the recorded condition

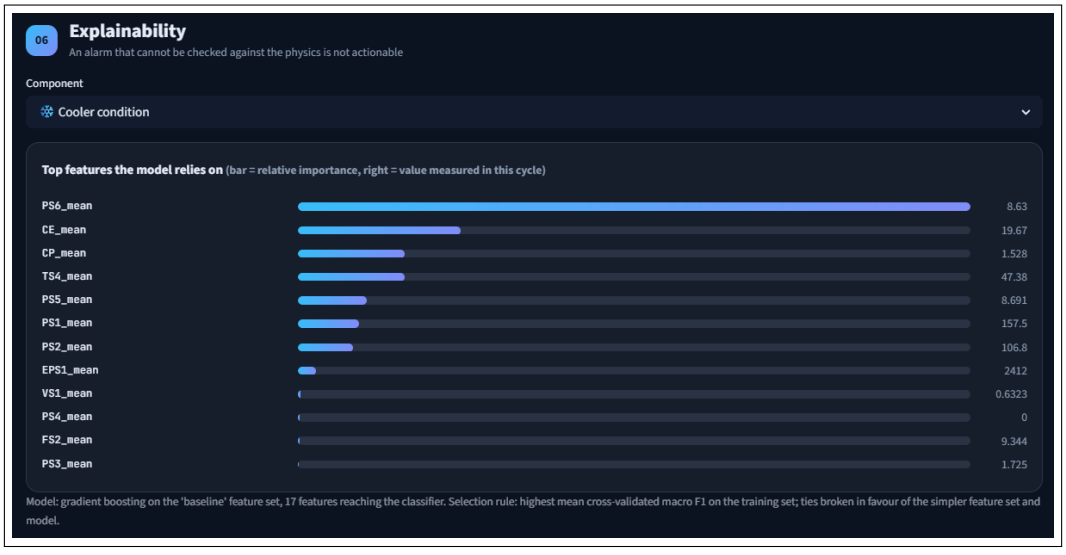


Figure 5.3: Dashboard: explainability panel for the cooler model

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This project built a predictive maintenance system for an industrial hydraulic rig, with feature engineering at its core. Each 60-second cycle of 43,680 raw values was converted into 626 interpretable features in five blocks—statistical, time-domain, rolling, frequency and hydraulic domain—then cleaned to 553 and reduced to 40 per model by a leakage-safe selection chain. The experiments show clearly what feature engineering contributes: logistic regression improved from 0.672 to 0.991 macro-F1 on the valve and from 0.642 to 0.977 on the accumulator, and the best models improved by 2–3 points on these two components. The final models achieved a held-out macro-F1 of 1.000 for the cooler, valve and pump and 0.988 for the accumulator, with more conservative blocked cross-validation estimates of 0.995, 0.984, 0.968 and 0.916. The most important features have direct physical meaning, and the dashboard makes predictions and their explanations accessible to engineers.

6.2 Future Scope

- Use blocked cross-validation as the model selection criterion, not only as a check.
- Apply feature selection to the accumulator model and use grouped importance to explain it better.
- Add features that capture transients around each load step, which may separate the two lowest accumulator levels.
- Validate the approach on data from other hydraulic machines and on gradual, in-service degradation.
- Collect run-to-failure data so that Remaining Useful Life can be estimated.

REFERENCES

[1] N. Helwig, E. Pignanelli, and A. Schütze, "Condition monitoring of a complex hydraulic system using multivariate statistics," in *Proc. IEEE Int. Instrum. Meas. Technol. Conf. (I2MTC)*, Pisa, Italy, 2015, pp. 210–215.

[2] N. Helwig, E. Pignanelli, and A. Schütze, "Condition monitoring of hydraulic systems," UCI Machine Learning Repository, 2015, doi: 10.24432/C5CW21.

[3] T. P. Carvalho, F. A. A. M. N. Soares, R. Vita, R. P. Francisco, J. P. Basto, and S. G. S. Alcalá, "A systematic literature review of machine learning methods applied to predictive maintenance," *Computers & Industrial Engineering*, vol. 137, Art. no. 106024, 2019.

[4] M. Christ, N. Braun, J. Neuffer, and A. W. Kempa-Liehr, "Time series feature extraction on basis of scalable hypothesis tests (tsfresh – A Python package)," *Neurocomputing*, vol. 307, pp. 72–77, 2018.

[5] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *Journal of Machine Learning Research*, vol. 3, pp. 1157–1182, 2003.

[6] A. Kraskov, H. Stögbauer, and P. Grassberger, "Estimating mutual information," *Physical Review E*, vol. 69, no. 6, Art. no. 066138, 2004.

[7] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[8] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[9] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[10] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 4, Art. no. 15, 2012.

[11] D. R. Roberts *et al.*, "Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure," *Ecography*, vol. 40, no. 8, pp. 913–929, 2017.